

5. Modelos lineares para classificação

No Capítulo 4, você aprendeu a utilizar modelos lineares para resolver problemas simples de regressão. Você também aprendeu como estender regressão linear para ajustar curvas complexas e construir redes neurais simples. Nesse Capítulo, desenvolveremos modelos lineares para o caso em que a variável de resposta y é uma *classe*, ao invés de um número real.

5.1 Regressão logística

Uma das maneiras mais naturais de criar um modelo de regressão consiste em escolher um modelo observacional para a variável de resposta y cujos parâmetros dependam diretamente do seu respectivo vetor de entradas $\mathbf{x}$. Por exemplo, na capítulo anterior, vimos que minimizar o MSE é equivalente a admitir uma verossimilhança da forma $y|\mathbf{x} \sim \mathcal{N}(\boldsymbol{\theta}^\top \mathbf{x}, \sigma^2)$, na qual o parâmetro de média é uma função linear de $\mathbf{x}$. Note também que essa escolha implica que y pode tomar valores arbitrários em $\mathbb{R}$, já que esse é o suporte da distribuição normal (i.e., região com densidade maior que zero).

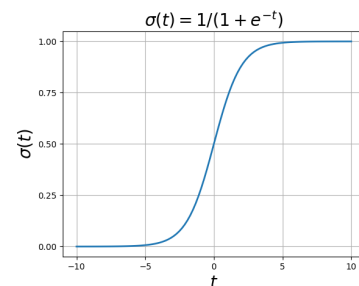
Se y é uma variável binária (0/1), a escolha mais comum é utilizar uma distribuição Bernoulli com parâmetro $r = g(\mathbf{x})$. Em outras palavras, falamos que y assume valor 1 com probabilidade r e 0 com probabilidade $1 - r$. Adotando esse modelo observacional para $y|\mathbf{x}$, resta-nos definir a função g para completar nosso modelo de regressão. Para tal, iremos calcular uma função linear de $\mathbf{x}$, como anteriormente, mas aplicaremos uma função que mapeie o valor resultante (comumente conhecido como logit) para $[0, 1]$, gerando valores válidos para a probabilidade r . Mais especificamente, usamos a função sigmoide $\sigma(t) = (1 + e^{-t})^{-1}$ para definir a probabilidade de $y|\mathbf{x}$ como:

$$p(y|\mathbf{x}) = \text{Ber}(y|\sigma(\boldsymbol{\theta}^\top \mathbf{x})) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})^y (1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}))^{1-y}. \quad (5.1)$$

Propriedades da função sigmoide. A Figura abaixo ilustra a função sigmoide. Note que ela é estritamente crescente, i.e., $t < t'$ implica $\sigma(t) < \sigma(t')$. Notavelmente, $\sigma(t)$ se aproxima de 0 quando t diminui e se aproxima de 1 quando t aumenta.

A função sigmoide também possui propriedades analíticas especialmente úteis computacionalmente, por exemplo:

$$\begin{aligned} \sigma(-t) &= 1 - \sigma(t), & \frac{d\sigma(t)}{dt} &= \sigma(t)\sigma(-t), \\ \sigma(t) &= \frac{1}{2} + \frac{1}{2} \tanh\left(\frac{t}{2}\right), & \int \sigma(t) dt &= \log \sigma(-t) + C. \end{aligned}$$



Dado um conjunto de treinamento $\mathcal{D}$ com N exemplos de treinamento $(\mathbf{x}_n, y_n)$, podemos então definir a função de verossimilhança $\mathcal{L}$ como

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^N \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i)^{y_i} (1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i))^{1-y_i}, \quad (5.2)$$

e otimizá-la para encontrar a estimativa de máxima verossimilhança para $\boldsymbol{\theta}$:

$$\begin{aligned} \hat{\boldsymbol{\theta}} &= \arg \max_{\boldsymbol{\theta} \in \mathbb{R}^{D+1}} \mathcal{L}(\boldsymbol{\theta}) \equiv \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{D+1}} -\log \mathcal{L}(\boldsymbol{\theta}) \\ &\equiv \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{D+1}} - \sum_{i=1}^N y_i \log \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i) + (1 - y_i) \log (1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i)) \\ &\equiv \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{D+1}} - \sum_{\substack{i=1 \dots N \\ y_i=1}} \log \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i) - \sum_{\substack{i=1 \dots N \\ y_i=0}} \log \sigma(-\boldsymbol{\theta}^\top \mathbf{x}_i) \end{aligned}$$

Similar ao MSE, que minimizamos no capítulo passado, a função objetivo $-\log \mathcal{L}$ é uma convexa. No entanto, não possuímos uma solução analítica para $\hat{\boldsymbol{\theta}}$ e, portanto, precisamos utilizar algum método de otimização numérica, como o SGD que já conhecemos. Para fins de implementação, podemos definir a variável $y'_i = 2y_i - 1$. Com isso, conseguimos escrever o gradiente $\nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta})$ da log verossimilhança para o i -ésimo exemplo de treinamento ℓ_i como:

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \log \sigma(y'_i \boldsymbol{\theta}^\top \mathbf{x}_i) \\ &= \frac{\partial \log \sigma(y'_i \boldsymbol{\theta}^\top \mathbf{x}_i)}{\partial \sigma(y'_i \boldsymbol{\theta}^\top \mathbf{x}_i)} \frac{\partial \sigma(y'_i \boldsymbol{\theta}^\top \mathbf{x}_i)}{\partial y'_i \boldsymbol{\theta}^\top \mathbf{x}_i} \frac{\partial y'_i \boldsymbol{\theta}^\top \mathbf{x}_i}{\partial \boldsymbol{\theta}} \\ &= \sigma(-y'_i \boldsymbol{\theta}^\top \mathbf{x}_i) y'_i \mathbf{x}_i \end{aligned}$$

Aplicando o algoritmo gradiente descendente à função custo da regressão logística, obtemos o seguinte algoritmo:

Algoritmo 1 Regressão Logística

- | | |
|--|---|
| 1: $\boldsymbol{\theta}^{(0)} \leftarrow \mathbf{0}$ | ▷ Inicializa os parâmetros/pesos. |
| 2: $\mathbf{y}' \leftarrow 2\mathbf{y} - 1$ | ▷ Transforma os rótulos para -1 e 1. |
| 3: for $t = 0, 1, 2 \dots$ do | |
| 4: $\mathbf{g} \leftarrow \sum_i \sigma(-y'_i \mathbf{x}_i^\top \boldsymbol{\theta}^{(t)}) y'_i \mathbf{x}_i$ | ▷ Computa o gradiente. |
| 5: $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} + \eta \mathbf{g}$ | ▷ Realiza a atualização dos parâmetros. |
| 6: Verifica a condição de parada. Por exemplo, se $\ \boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^{(t)}\ < 0.000001$. | |
| 7: end for | |
| 8: return $\boldsymbol{\theta}$ | ▷ Retorna os parâmetros/pesos finais. |
-

Interpretação geométrica. Uma vez que obtivemos $\hat{\boldsymbol{\theta}}$, podemos estimar a probabilidade de uma nova amostra $\mathbf{x}^*$ pertencer à classe 1 como $\sigma(\hat{\boldsymbol{\theta}}^\top \mathbf{x}^*)$ e a de pertencer à classe 0 como $1 - \sigma(\hat{\boldsymbol{\theta}}^\top \mathbf{x}^*)$. Com isso em mente, se precisamos prever a classe de $\mathbf{x}^*$, é razoável escolher aquela que achamos mais provável. Lembre que $\sigma(0) = 0.5$. Portanto, o plano $\hat{\boldsymbol{\theta}}^\top \mathbf{x} = 0$ caracteriza os pontos $\mathbf{x} \in \mathcal{X}$ que cremos ter probabilidade idêntica de pertencer a ambas as classes. Isso implica que todos os vetores de entrada cujo ângulo γ com o vetor

normal $\hat{\theta}$ é menor que noventa graus são classificados como positivos (classe 1) — lembre que $\hat{\theta}^\top \mathbf{x} = \|\hat{\theta}\|_2 \|\mathbf{x}\|_2 \cos \gamma$. Os demais pontos são classificados como negativos (classe 0).

Convexidade do problema de aprendizado. Como a soma de funções convexas é também convexa, basta verificar que as $\ell_1, \dots, \ell_N$ são convexas para provarmos que $-\log \mathcal{L}$ também o é. Para esse fim, podemos usar o fato de que a função composta $h = g \circ f$ é convexa se f é côncava e g é convexa não-crescente. Note que $y_i' \theta^\top \mathbf{x}_i$ é tanto côncava como convexa, como é o caso de funções lineares. Em contrapartida, $-\log \sigma(t)$ é convexa não-crescente já que ela decresce com t e sua derivada $-\sigma(-t)$ é estritamente crescente.

Entropia cruzada binária. Na comunidade de ML, é comum se referir ao logaritmo negativo da verossimilhança Bernoulli como entropia cruzada binária (*binary cross entropy*, BCE). De forma geral, a entropia cruzada entre duas funções de massa/densidade p e q sobre a mesma variável aleatória z e com suportes idênticos é definida como:

$$H(p, q) = \mathbb{E}_{z \sim p} \left[\log \frac{1}{q(z)} \right],$$

e dá-se o nome BCE para o caso especial em que p e q são distribuições Bernoulli.

Em teoria da informação, é comum interpretar $\log 1/q(z)$ como uma medida de *surpresa*, i.e., do quanto observar um valor específico z contrasta com seu conhecimento prévio, representado por q . Nesse contexto, $H(p, q)$ é o valor dessa medida se os valores de z são amostrados de p (ao invés de q). Vale ressaltar que, para um p fixo, $q = p$ minimiza $H(p, q)$. Nesse caso, a quantia $H(p) := H(p, p)$ é chamada de entropia. Por sua vez, a entropia também pode ser vista como uma medida de concentração de p , atingindo seu valor máximo quando p é uma distribuição uniforme.

Para concluir que a BCE generaliza $-\log \text{Ber}(y|r)$, basta definir $p(z) = \text{Ber}(z|y)$ e tomar $q(z) = \text{Ber}(z|r)$. Com essas escolhas, obtemos:

$$\begin{aligned} H(p, q) &= -y \log q(1) - (1 - y) \log q(0) \\ &= -(y \log r + (1 - y) \log (1 - r)) \end{aligned}$$

o que implica que:

$$e^{-H(p, q)} = r^y (1 - r)^{(1-y)} = \text{Ber}(y|r)$$

5.2 Regressão logística Bayesiana

Na seção anterior, discutimos como obter uma estimativa pontual $\hat{\theta}$ para θ via máxima verossimilhança (MLE). Um problema com estimativas pontuais, no entanto, é que elas não refletem nossa incerteza sobre o valor estimado. Intuitivamente, por exemplo, esperamos que estimativas feitas com grandes quantidades de dados sejam mais confiáveis que as feitas com poucos dados. Para observar que MLE não captura esse aspecto, basta notar que repetir o conjunto de dados qualquer número arbitrário de vezes não causa impacto algum em $\hat{\theta}$.

A estatística Bayesiana propõe uma saída intuitiva para esse empasse. A ideia é modelar os parâmetros θ do modelo como uma variável aleatória, usando uma distribuição *a priori* $p(\theta)$ e

aplicar a regra de Bayes para computar a distribuição de θ condicionada nas observações $\mathcal{D}$, que chamamos de *posteriori*:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{\int_{\theta} p(\mathcal{D}|\theta)p(\theta)}$$

Informalmente, a posteriori representa a incerteza que temos sobre o valor da variável θ . Além disso, vale ressaltar que a priori nos permite encapsular conhecimento prévio, potencialmente subjetivo, sobre θ (i.e., antes de ver os dados) e sua escolha é uma questão de modelagem estatística. Quando não possuímos informação significativa sobre os dados, é comum escolher uma distribuição de alta entropia como priori. No caso em que θ assume valores reais, e.g., poderíamos usar uma priori Gaussiana com alta variância.

No caso da regressão logística, é comum adotar uma priori Gaussiana sobre θ e a verossimilhança que usamos na seção anterior, resultando no modelo:

$$\theta \sim \mathcal{N}(\mu, \Sigma)$$

$$y_n | \mathbf{x}_n \sim \text{Ber}(\sigma(\theta^\top \mathbf{x}_n)) \forall n = 1 \dots N,$$

cujas posteriori é dada por

$$p(\theta|\mathcal{D}) = \frac{\prod_{i=1}^N \text{Ber}(y_i | \sigma(\theta^\top \mathbf{x}_i)) \mathcal{N}(\theta | \mu, \Sigma)}{\int_{\theta \in \mathbb{R}^{D+1}} \prod_{i=1}^N \text{Ber}(y_i | \sigma(\theta^\top \mathbf{x}_i)) \mathcal{N}(\theta | \mu, \Sigma)} \quad (5.3)$$

Notavelmente, computar a posteriori acima depende da resolução de uma integral que não possui forma fechada. Consequentemente, também não há uma forma analítica para $p(\theta|\mathcal{D})$. Essa dificuldade técnica não é uma raridade em modelos Bayesianos. Apesar de algumas escolhas pareadas de verossimilhança e priori resultarem em posteriores com forma analítica, esse não é o caso geral. Para driblar esse problema, usaremos métodos numéricos para aproximar a posteriori com distribuição mais simples, de forma conhecida.

Previendo a label y^* para um novo input $\mathbf{x}^*$. Suponha que conseguimos computar a posteriori $p(\theta|\mathcal{D})$, como podemos obter uma distribuição para $p(y^*|\mathbf{x}^*)$? Quando estavamos usando uma estimativa pontual $\hat{\theta}$ (MLE), obtivemos uma distribuição sobre y^* simplesmente encaixando $\hat{\theta}$ na nossa verossimilhança, i.e., tomamos $p(y^*|\mathbf{x}^*) \approx \text{Ber}(\sigma(\hat{\theta}^\top \mathbf{x}^*))$. No paradigma Bayesiano, levamos em consideração a incerteza sobre θ (codificada em nossa posteriori), ponderando cada valoração de θ pela sua densidade posteriori. O resultado, é o que chamamos de posteriori preditiva:

$$p(y^*|\mathbf{x}^*) = \int_{\theta \in \mathbb{R}^{D+1}} p(y^*|\mathbf{x}^*, \theta) p(\theta|\mathcal{D})$$

Máxima verossimilhança e máximo a posteriori. Uma das maiores virtudes do paradigma Bayesiano é oferecer uma maneira de quantificar incerteza. No entanto, há situações nas quais computar a posteriori, mesmo que de maneira aproximada, pode se tornar computacionalmente indesejável. Nesses casos, é comum procurar o ponto que maximiza a posteriori e tomá-lo como estimativa pontual. Chama-se esse procedimento de máximo a

posteriori (MAP). Mais concretamente, a estimativa $\hat{\theta}_{\text{MAP}}$ pode ser escrita obtida como:

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg \max_{\theta} \log p(\theta|\mathcal{D}) \\ &= \arg \max_{\theta} \log p(\mathcal{D}|\theta) + \log p(\theta) - \log \int_{\theta} p(\mathcal{D}|\theta)p(\theta) \\ &= \arg \max_{\theta} \log p(\mathcal{D}|\theta) + \log p(\theta)\end{aligned}$$

e, portanto, pode ser interpretada como uma versão *regularizada* do MLE, na qual $\log p(\theta)$ penaliza regiões pouco prováveis a priori.

Exemplo 5.1 — Priori conjugada. Para escolhas específicas de priori e verossimilhança, a distribuição posteriori possui forma analítica. Uma instância dessas ocorre quando $p(\theta)$ é uma distribuição Beta e a verossimilhança $p(\mathcal{D}|\theta)$ é Bernoulli. Mais concretamente, suponha que escolhemos uma priori $\text{Beta}(\alpha, \beta)$ para θ , dada por:

$$\text{Beta}(\theta|\alpha, \beta) = \frac{\theta^{\alpha-1} \theta^{\beta-1}}{B(\alpha, \beta)},$$

onde $B(\alpha, \beta) = \int_{\theta \in [0,1]} \theta^{\alpha-1} \theta^{\beta-1}$ é uma constante normalizadora.

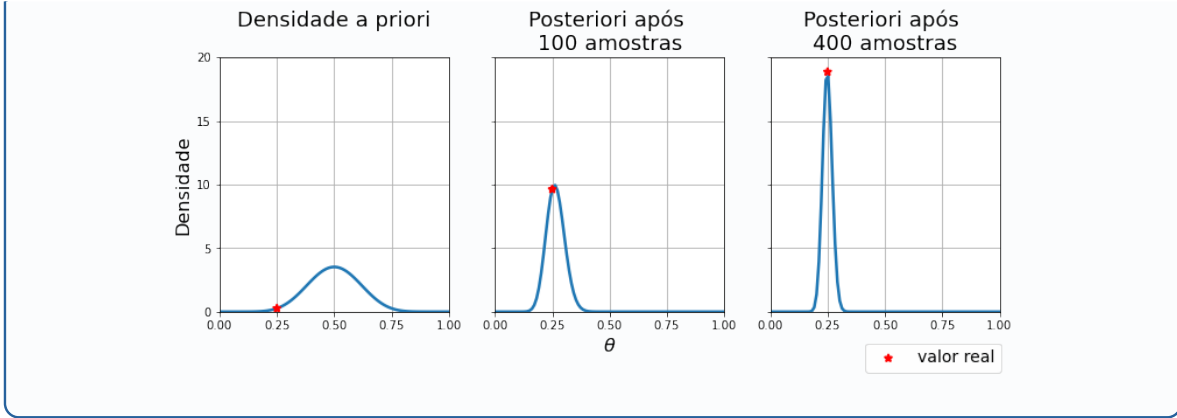
Podemos inferir, então, o seguinte sobre a posteriori do nosso modelo:

$$\begin{aligned}p(\theta|\mathcal{D}) &\propto \prod_{n=1}^N \text{Ber}(y_n|\theta) \text{Beta}(\theta|\alpha, \beta) \\ &\propto \prod_{n=1}^N \theta^{y_n} (1-\theta)^{1-y_n} \theta^{\alpha-1} \theta^{\beta-1} \\ &\propto \theta^{\sum_{n=1}^N y_n} (1-\theta)^{N-\sum_{n=1}^N y_n} \theta^{\alpha-1} \theta^{\beta-1} \\ &\propto \theta^{\sum_{n=1}^N y_n + \alpha - 1} (1-\theta)^{N - \sum_{n=1}^N y_n + \beta - 1} \\ &\propto \theta^{\alpha^* - 1} (1-\theta)^{\beta^* - 1}\end{aligned}$$

onde $\alpha^* = \sum_{n=1}^N y_n + \alpha$ e $\beta^* = N - \sum_{n=1}^N y_n + \beta$.

Note que $p(\theta|\mathcal{D})$ é proporcional à uma distribuição beta com parâmetros α^* e β^* . Portanto, concluímos que nossa posteriori é uma $\text{Beta}(\alpha^*, \beta^*)$. Chamamos casos como esse, no qual a posteriori pertence à mesma família da priori, de priori conjugada.

Para ilustrar o uso da regra de Bayes, a figura abaixo mostra atualizações da posteriori derivada acima para diferentes números de amostras N . Para tal, assumimos que a distribuição geradora dos dados é $\text{Ber}(0.25)$ e usamos uma priori $\text{Beta}(10, 10)$. Veja que, à medida que vemos mais amostras, a posteriori se afunila ao redor de 0.25.



5.2.1 Aproximação de Laplace

A aproximação de Laplace é, possivelmente, a mais simples técnica de inferência Bayesiana aproximada. A ideia é construir uma aproximação simples $q(\boldsymbol{\theta})$ para a posteriori $p(\boldsymbol{\theta}|\mathcal{D})$ usando uma expansão de Taylor de segunda ordem em $\log p(\boldsymbol{\theta}|\mathcal{D})$ ao redor da moda $\mathbf{m}$ da posteriori (i.e., o ponto de máxima densidade):

$$\log p(\boldsymbol{\theta}|\mathcal{D}) \approx \log p(\mathbf{m}|\mathcal{D}) + (\boldsymbol{\theta} - \mathbf{m})^\top \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}|\mathcal{D})|_{\boldsymbol{\theta}=\mathbf{m}} + \frac{1}{2} (\boldsymbol{\theta} - \mathbf{m})^\top \nabla_{\boldsymbol{\theta}}^2 \log p(\boldsymbol{\theta}|\mathcal{D})|_{\boldsymbol{\theta}=\mathbf{m}} (\boldsymbol{\theta} - \mathbf{m}),$$

onde ∇ denota o vetor gradiente e ∇^2 denota a matriz de segundas derivadas (Hessiana). Note que $\log p(\mathbf{m}|\mathcal{D})$ é uma constante com respeito a $\boldsymbol{\theta}$ e lembre que o gradiente de uma função em sua moda, caso ela exista, é zero. Então, concluímos que

$$\log p(\boldsymbol{\theta}|\mathcal{D}) \approx -\frac{1}{2} (\boldsymbol{\theta} - \mathbf{m})^\top \mathbf{H} (\boldsymbol{\theta} - \mathbf{m}) + \text{constante},$$

onde $\mathbf{H} = \nabla_{\boldsymbol{\theta}}^2 - \log p(\boldsymbol{\theta}|\mathcal{D})|_{\boldsymbol{\theta}=\mathbf{m}}$. Por *design*, construímos q tal que $\log q$ difira da expansão acima apenas por uma constante aditiva, então:

$$\log q(\boldsymbol{\theta}) = -\frac{1}{2} (\boldsymbol{\theta} - \mathbf{m})^\top \mathbf{H} (\boldsymbol{\theta} - \mathbf{m}) + \text{constante} \Rightarrow q(\boldsymbol{\theta}) \propto e^{-\frac{1}{2} (\boldsymbol{\theta} - \mathbf{m})^\top \mathbf{H} (\boldsymbol{\theta} - \mathbf{m})},$$

e como $q(\boldsymbol{\theta})$ é proporcional á uma densidade normal multivariada com média $\mathbf{m}$ e matrix de covariância igual á inversa de $\mathbf{H}$, temos

$$q(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta} | \mu = \mathbf{m}, \Sigma = \mathbf{H}^{-1}), \text{ onde } \mathbf{m} = \arg \max_{\boldsymbol{\theta} \in \Theta} p(\boldsymbol{\theta}|\mathcal{D}) \text{ e } \mathbf{H} = \nabla_{\boldsymbol{\theta}}^2 - \log p(\boldsymbol{\theta}|\mathcal{D})|_{\boldsymbol{\theta}=\mathbf{m}}$$

Note que o procedimento acima envolve inverter $\mathbf{H}$. Como $\mathbf{m}$ é um mínimo local para a função $-\log p(\cdot|\mathcal{D})$, segue diretamente das condições de optimalidade de segunda ordem que $\mathbf{H}$ é PSD. Caso $\mathbf{H}$ não seja PD ou haja instabilidade numérica na inversão de $\mathbf{H}$, uma prática comum é adicionar um pequeno valor $c > 0$ à sua diagonal.

Para aplicar o método de Laplace ao nosso modelo de regressão logística Bayesiano (Equação 5.3), podemos usar alguma variação de gradiente descendente para achar $\mathbf{m}$ e, assumindo $\boldsymbol{\mu} = \mathbf{0}$ e $\Sigma = cI$ para $c > 0$, as entradas $H_{i,j}$ da Hessiana $\mathbf{H}$ são dadas por:

$$H_{i,j} = \begin{cases} \sum_{n=1}^N \sigma(\boldsymbol{\theta}^\top \mathbf{x}_n) \sigma(-\boldsymbol{\theta}^\top \mathbf{x}_n) x_{n,i} x_{n,j} & \text{if } i \neq j, \\ \sum_{n=1}^N \sigma(\boldsymbol{\theta}^\top \mathbf{x}_n) \sigma(-\boldsymbol{\theta}^\top \mathbf{x}_n) x_{n,i} x_{n,j} + c^{-1} & \text{if } i = j. \end{cases}$$

5.3 Extensão para problemas multiclasse: classificador *softmax*

Nesse capítulo, nos focamos em problemas de classificação binária, em que $|\mathcal{Y}| = 2$. No entanto, é fácil generalizar as técnicas que discutimos para problemas multi-classe (i.e., $|\mathcal{Y}| > 2$). Para tal, basta substituir o nosso modelo observacional Bernoulli por uma distribuição categórica. Lembre que a Bernoulli é parametrizada por um parâmetro escalar que dita a probabilidade de cada classe. No caso da categórica, precisamos de um vetor de probabilidades, i.e., um vetor $\mathbf{r}$ de tamanho $L = |\mathcal{Y}|$, em que cada entrada r_l denota a probabilidade de classe l . Naturalmente, todas as entradas de $\mathbf{r}$ devem ser não-negativas e $\sum_{l=1}^L r_l = 1$. Resta-nos, então, expressar $\mathbf{r}$ como uma função de $\mathbf{x}$. Para tal, podemos generalizar nosso procedimento que usamos para regressão logística.

Primeiro, calculamos um vetor de logits $\mathbf{z}$, desta vez usando uma transformação linear para cada uma das L classes:

$$\mathbf{z} = \begin{bmatrix} \mathbf{x}^\top \boldsymbol{\theta}^{(1)} \\ \mathbf{x}^\top \boldsymbol{\theta}^{(2)} \\ \vdots \\ \mathbf{x}^\top \boldsymbol{\theta}^{(L)} \end{bmatrix},$$

Finalmente, aplicamos a função Softmax para transformar $\mathbf{z}$ em um vetor de probabilidades e obter $\mathbf{r}$, que é dado por:

$$\mathbf{r} = \text{Softmax}(\mathbf{z}) = \frac{e^{\mathbf{z}}}{\sum_{l=1}^L e_l^{\mathbf{z}}},$$

onde $e^{\mathbf{z}}$ é um abuso de notação denotando exponenciar cada uma das entradas de $\mathbf{z}$.

Bibliografia

- Ahn, K., Yun, C., & Sra, S. (2020). SGD with shuffling: optimal rates without component convexity and large epoch requirements. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Atkeson, C. G., Moore, A. W., & Schaal, S. (1997). Locally Weighted Learning. *Artificial Intelligence Review*, 11(1–5) (ver página 20).
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press. (Ver página 22).
- Broomhead, D., & Lowe, D. (1988). Multivariable Functional Interpolation and Adaptive Networks. *Complex Systems*, 2, 321–355 (ver página 30).
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27 (ver página 19).
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4), 303–314 (ver página 59).
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14, 179–211 (ver página 64).
- Fukushima, K. (1975). Cognitron: A Self-Organizing Multilayer Neural Network. *Biological Cybernetics*, 20, 121–136 (ver página 59).
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366 (ver página 59).
- Lloyd, S. P. (1982). Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28, 129–137 (ver página 30).
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press. (Ver página 22).
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133 (ver páginas 5, 56).
- Micchelli, C. A. (1986). Interpolation of scattered data: Distance matrices and conditionally positive definite functions. *Constr. Approx.*, 2, 11–22 (ver página 30).
- Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. (Ver página 57).
- Mohamed, S., Rosca, M., Figurnov, M., & Mnih, A. (2020). Monte Carlo Gradient Estimation in Machine Learning. *Journal of Machine Learning Research (JMLR)* (ver página 42).
- Nair, V., & Hinton, G. (2010). Rectified linear units improve restricted boltzmann machines. *International conference on machine learning (ICML)* (ver página 59).
- Nguyen, L., Tran-Dinh, Q., Phan, D., Nguyen, P., & van Djik, M. (2021). A Unified Convergence Analysis for Shuffling-Type Gradient Methods. *Journal of machine learning research (JMLR)* (ver página 11).
- Novikoff, A. B. (1962). On Convergence Proofs on Perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622 (ver página 57).

- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT Press. (Ver página 52).
- Rosenblatt, F. (1957). *The perceptron: A perceiving and recognizing automaton (Project PARA)* (relatório técnico). Cornell University (Technical Report 85-460-1). (Ver página 56).
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088), 533–536 (ver páginas 57, 59).
- Safran, I., & Shamir, O. (2021). Random Shuffling Beats SGD Only After Many Epochs on Ill-Conditioned Problems. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press. (Ver página 44).
- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, 49, 433–460 (ver página 5).
- Weinberger, K. Q., Blitzer, J., & Lawrence, S. K. (2006). Distance Metric Learning for Large Margin Nearest Neighbor Classification. Em Y. Weiss, B. Schölkopf & J. C. Platt (Editores), *Advances in Neural Information Processing Systems 18* (páginas 1473–1480). MIT Press. (Ver página 22).
- Widrow, B., & Lehr, M. (1990). 30 years of adaptive neural networks: perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9), 1415–1442. <https://doi.org/10.1109/5.58323> (ver página 57).
- Xu, M., Quiroz, M., Kohn, R., & Sisson, S. A. (2019). Variance reduction properties of the reparameterization trick. *Artificial Intelligence and Statistics (AISTATS)* (ver página 42).